

BUILD A LARGE LANGUAGE MODEL

Capítulo 2 - Working with text data (Trabalhando com dados de texto)

Anotações:

- Foca na implementação da pipeline de amostragem de dados. Dividir o texto em palavras individuais ou tokens de subpalavras.
- Representações vetoriais das palavras: LLMs não podem processar textos diretamente. O texto é categórico e não é compatível com operações matemáticas que usamos para redes neurais, por isso precisamos representar as palavras em vetores de valores contínuos.
- Embedding (incorporar): converter dados em vetores. Usando uma camada de rede neural ou um modelo pré-treinado de rede neural, nós podemos incorporar diferentes tipos de dados. Diferentes tipos de dados requerem diferentes modelos de embedding (um de vídeo não é compatível com um de texto). Ele é um mapeamento a partir de objetos discretos (texto, vídeo, imagens) em pontos em um vetor contínuo. O propósito geral deles é converter não numéricos dados em um formato que a rede neural pode processar. Os embeddings de texto são os mais comuns, mas existe também de frases, parágrafos e documentos inteiros.
- Geração aumentada por recuperação (retrieval-augmented generation): Mais utilizado por embeddings de frases e parágrafos. Combina geração (produção de texto) com recuperação (buscar uma base de conhecimento externa) para puxar informações relevantes quando gerar texto. Mas o foco é em embeddings de palavra (gerar o texto uma palavra por vez)
- Word2Vec: rede neural treinada para gerar embeddings word por meio de previsão do contexto de uma palavra a partir de uma palavra-alvo ou vice-versa. A ideia é que palavras que aparecem em contextos similares tendem a ter significados similares. Quando projetado em embeddings bidimensionais termos semelhantes são agrupados. Podemos pré treinar esse modelo para gerar embeddings para modelos de machine learning.
- Word embeddings: Podem ter variadas dimensões (1 a milhares). Dimensões maiores podem capturar mais nuances de relacionamentos mas ao custo da eficiência do computador. Geralmente em vetores de 2 dimensões podemos plotar em um gráfico e observar que conceitos similares tendem a ficarem próximos em espaços de embedding.
- LLMs tem o costume de produzir seus próprios embeddings que são parte da camada de entrada e são atualizados durante treinamento. A vantagem de fazer isso, ao invés de utilizar Word2Vec é que os embeddings são otimizados para tarefas específicas e dados. LLMs podem criar embeddings de saídas contextualizadas.

- Dimensões grandes apresentam desafios para a visualização devido a nossa percepção. E os gráficos são limitados a 3 dimensões. Mas mesmo assim, utilizamos dimensões maiores em LLMs.
- Tokenizing text: como separar o texto de entrada em tokens individuais, esse é um pré processo que é requerido para criar embeddings para a LLM. Os tokens podem ser palavras individuais ou caracteres especiais. Texto de entrada -> Texto tokenizado -> IDs do tokens -> Tokens embedding -> Transformer decoder GPT-like -> Etapas de pós processamento -> Texto de saída
- Tokens IDs: Converter cada string(token) em uma representação numérica inteira para produzir o ID de cada token. Essa conversão é o passo intermediário antes de converter os ID em embedding vetores. Precisamos criar um vocabulário que define como mapeamos cada palavra/caractere especial único para um inteiro único
- Vocabulário: construído ao tokenizar o texto inteiro em um conjunto de dados em treinamentos para tokens individuais, tokens duplicados são removidos, são arranjados em ordem alfabética. Os tokens únicos são adicionados no vocabulário que define um mapeamento para cada token com um valor inteiro único. Usa os vocabulários para converter os textos tokens em token IDs.
- Vocabulário inverso: Quando queremos converter as saídas de uma LLM de números para texto, ou seja, tokens IDs para texto, criamos a versão inversa do vocabulário que mapeia os IDs de volta para seus textos correspondentes.
- Implementações de tokenização possuem 2 métodos: codificador (pega o texto de exemplo e divide em tokens individuais para depois converter em tokens IDs pelo vocabulário) e o decodificador(pega o tokens IDs e converte de volta para em tokens de texto e concatena em um texto corrido).
- Precisamos considerar um set de treinamento diverso e grande, uma vez que se a palavra não está nele ela não está no vocabulário e aí o código gera erro em encoder/decoder.
- Precisamos modificar o vocabulário e o tokenizer para conseguir lidar com palavras desconhecidas (tokens especiais de contexto -> marcadores para palavras desconhecidas e limites do documento). Colocamos tokens especiais junto no vocabulário para lidar com certos contextos, como <|unk|> para representar novas palavras e <|endoftext|> para separar duas fontes de texto sem relação (colocando um token desses antes de cada documento/livro que segue a fonte de texto anterior, isso ajuda a LLM a entender que mesmo que os textos estejam concatenados para treinamento, eles não se relacionam).
- Outros tokens especiais: BOS -> Marca início do texto, EOS -> Marca fim do texto, bem similar do endoftext, PAD -. Ao lidar com LLMs com batches maiores que um o batch pode conter textos de tamanhos variados, para garantir que os textos tenham o mesmo tamanho

utilizamos esse token para estender os textos mais curtos para o tamanho do texto mais longo no batch.

- GPT: Não utiliza nenhum desses outros tokens especiais, apenas o endoftext que também é usado para fazer o que o pad faz, quando treinando batch de entrada usamos uma máscara (não atende aos tokens PAD). Também, não utiliza o unk, visto que o GPT usa um token byte pair encoding (codificação de par de byte) que quebra as palavras em unidades de subpalavras.
- Byte pair encoding (BPE): esquema sofisticado de tokenização, usado para treinar LLMs como o GPT, implementar pode ser bem complicado então podemos utilizar uma biblioteca open source chamada TIKTOKEN que implementa o algoritmo BPE de forma eficiente. Usar esse tokenizer é similar às classes de tokenização, também utiliza codificador e decodificador. No GPT o tamanho do vocabulário o endoftext é o maior ID token. O BPE codifica e decodifica palavras desconhecidas sem usar o unk, o algoritmo dele quebra as palavras em sub palavras ou caracteres individuais, permitindo lidar com palavras fora do vocabulário, assim consegue analisar qualquer palavra sem precisar substituí-las com unk. O algoritmo constrói o vocabulário ao mesclar iterativamente caracteres frequentes em sub palavras e frequentes sub palavras em palavras, começa adicionando todos os caracteres individuais ao vocabulário e mescla combinações que frequentemente ocorrem juntas ($d + e = de$), as mesclagens são determinadas pela frequência de corte.
- Embeddings: Amostragem de dados com janela deslizante, gerar os pares de entrada-alvo necessários para treinar uma LLM. LLM são pré treinadas ao prever a próxima palavra em um texto. Extrair blocos de entrada com sub amostras que servem de entrada para a LLM, a tarefa de treinamento de predição da próxima palavra segue o bloco de entrada, durante o treinamento mascaramos todas as palavras que passaram o alvo.
- Entradas-alvo: Primeiro criamos os pares de entradas-alvo com BPE. Segundo, implementamos um dataloader eficiente que itera com o dataset de entrada e retorna as entradas e alvos como tensores, nesse caso 2 tensores, o tensor de entrada que contém o texto que o LLM vê e o tensor de alvo que inclui os alvos para o LLM prever, o código opera em tokens IDs visto que o método de codificação do BPE consegue tokenizar o texto e converter em tokens IDs em apenas um passo.
- Alvo (target): entradas transferidas por 1 posição, vão para frente por assim dizer.
- Batches pequenos requerem menos memória durante treinamento mas tem mais atualizações barulhentas do modelo. Batch é um hiperparâmetro.
- Mesmo que o stride seja maior que 1, não pulamos nenhuma palavra porque evita qualquer sobreposição entre os batches, uma vez que uma maior sobreposição poderia levar a um aumento de overfitting.

- O último passo para preparar o texto de entrada para o treinamento LLM é converter os tokens IDs em embedding vetores. A representação de vetores contínuos (embeddings) é necessária para LLM estilo GPT, visto que são redes neurais profundas treinadas com algoritmo de retropropagação.
- Camada embedding: é essencialmente uma operação de busca que recupera linhas da matriz de pesos da camada de embedding por meio de um token ID. É uma maneira mais eficiente de implementar a codificação one-hot seguida pela multiplicação de matrizes em uma camada totalmente conectada.
- Embeddings: Transforma palavras em vetores, cada token recebe um ID e esse ID vira um vetor. O problema é que o embedding não sabe onde a palavra está na frase e ordem importa na linguagem. O self-attention consegue olhar para todos os tokens e descobrir relações entre eles, mas não sabe naturalmente onde é a posição de uma palavra. É aí que vem o positional encoding / positional embedding.
- Camadas embeddings: realizam uma operação de busca, recuperando o vetor de embedding correspondente ao ID do token na matriz de pesos da camada de embedding. Essa camada converte um ID de token na mesma representação vetorial, independentemente de onde ele esteja localizado na sequência de entrada.
- Codificando posição da palavra: “O gato comeu” -> [5, 6, 12] cada número é um token ID -> Cada número o embedding transforma em um vetor 5 = [0.2, 0.7, ...] -> Existe um problema de que o embedding não sabe onde cada palavra está na frase, visto o token 5 recebe exatamente o mesmo vetor independente se está [5, 6, 12] ou [12, 6, 5], o embedding responde qual é o token mas não em qual posição ele está e a ordem das palavras importa muito e o mecanismo de self-attention também não sabe naturalmente onde está o token -> Solução é Positional Embeddings, onde adicionamos ao embedding da palavra uma informação dizendo onde ela está, por exemplo token 2 “gato” embedding [0.3, 0.7, ...] posição 2 positional [0.1, 0.3, ...] e o embedding final [0.4, 1.0, ...], assim o modelo recebe 2 informações qual é a palavra e onde ela está.
- Absolute positional embeddings: informa a posição exata, ou seja, cada posição possui seu próprio vetor.
- Relative positional embeddings: Em vez de focar apenas na posição absoluta, representam a relação entre as posições, por exemplo “gato” está 2 posições depois de “O”. A vantagem aqui é que o modelo consegue generalizar melhor em sequências de tamanhos diferentes, mesmo que não tenha visto que tal tamanho no treinamento.
- Ambos têm como objetivo aumentar a capacidade das LLMs de entender a ordem e relações entre tokens. A escolha entre os 2 tipos depende da aplicação e da natureza dos dados.
- Stride menor = mais sobreposição.

- O token embedding diz “o que é cada token”, positional embedding diz “onde esse token está” e os dois são somados para que a LLM tenha as duas informações (input_embeddings). Os input_embeddings que criamos são os exemplos de entrada transformados em vetores. Agora eles podem ser processados pelos principais módulos do LLM.